



Modern Data Analytics

Martial Luyts

Catholic University of Leuven, Belgium

`martial.luyts@kuleuven.be`



LEUVEN STATISTICS
RESEARCH CENTRE

Contents

0. Introduction	1
0.1. Goal & organisation of the course	2
0.2. Scheme	4
0.3. Evaluation & deadlines	5
0.4. Use of GenAI for this course	8
0.5. Questions?	10
1. Python Fundamentals for Data Scientists	11
1.1. Python	12

1.2. Writing Code	18
1.3. Google Colab	31
1.4. Git, Github & Hugging Face	38
1.5. Kaggle	47
1.6. Virtual Environments	50
1.7. Object Oriented Programming	56

Part 0:

Introduction

0.1 Goal & organisation of the course

In the course of MDA, you will

- Acquire the necessary practical skill set and theoretical knowledge to deal with a wide variety of data science tasks.
- Be exposed to a solid set of tools to successfully approach machine learning (ML) problems.
- Focus on modeling knowledge, as well as knowledge related to setting up the required IT infrastructure to execute ML projects in practice.

Organisation:

- **Traditional on-campus lecture format** in which a lecture introduces the material to be studied by you.
- After class, recording of the lecture will become available on Toledo.
- Each lecture is related to a specific theme of the course (see Section 0.2).
- Throughout the course, examples of implementations will be given using the **Python programming language** or the **AWS cloud environment**.
- To successfully complete the course, you will need to be able to execute and program Python code in both Jupyter Notebook and Integrated Development Environment (IDE). Version of Python that will be used is 3.12.

0.2 Scheme

Date	Content	Teacher
<u>Part 1: Python Fundamentals for Data Scientists</u>		
10/02	Introduction to Python and Object Oriented Programming	Martial Luyts
17/02	Data Science in Python: Packages	Ruben Kerkhofs
25/02	Data Science Algorithms in Python	Gregory van Kruijsdijk
03/03	Machine Learning Pipelines in Scikit Learn	Gregory van Kruijsdijk
<u>Part 2: Dashboarding in Python</u>		
10/03	Dashboarding in Python: Shiny for Python	Martial Luyts
17/03	Q&A session 1	
<u>Part 3: IT infrastructure for Data Science and Analytics</u>		
24/03	Key Components of an Analytics Infrastructure	Ruben Kerkhofs
31/03	Code Collaboration and Versioning	Gregory van Kruijsdijk
21/04	Containerisation using Docker	Martial Luyts
28/04	Cloud Computing 1	Martial Luyts
05/05	Cloud Computing 2	Martial Luyts
12/05	Data Processing Analytics, Scalability, and Performance	Ruben Kerkhofs
19/05	Q&A session 2	

0.3 Evaluations & deadlines

The evaluation of MDA is based on three components:

1. A **group report** which consists of a description of your group project. This group project is submitted on a per-group basis meaning that there is only a single submission per group. (2pt)
2. A **group project** in which you work together with your group to solve a data science project which will be presented during an **oral examination** (in group). (6pt)
3. Your **individual questions** asked to you during the group project presentation. Each student should expect to receive questions on both the group project content as well as any of the course material presented during the lectures. (12pt)

!!Important: The group project is counted for the full 6 points on the condition that at least 6/12 is obtained for the individual questions. In case the score for the individual questions is less than 6/12, the score for the report is reduced to at most 4/6. In case the score for the individual questions is less than 3/12, the score for the report is reduced to at most 2/6. This is to encourage active participation in the team work.

The following deadlines needs to be respected:

- 03/03: Group composition
- 24/03: Project proposal
- 25/05: Project submission

!!Important: These deadlines needs to be respected. Timely submission of the report is a necessary condition to take part in the exam. In case the deadline is not met, the score for this course will be NA (Not Attended).

!!Important: Presentations of the group project will be planned outside the exam period. Details about the date will be follow soon on Toledo!

0.4 Use of GenAI for this course

- Permitted:
 - **Improve language and text**, including spelling, grammar, and writing style.
 - **Correcting and structuring code** that has been **written by the student themselves**, without adding new functionalities or solutions generated by AI.
- Not permitted:
 - Generate complete texts, answers, or solutions without **critical evaluation and adaptation** by the student.
 - Writing substantial new code or solving programming tasks without clear, demonstrable student input.
 - Submitting AI-generated content as if it were solely the student's own original work.

!!Important: You (as student) remain **fully responsible** for the accuracy and originality of ALL submitted work.

- Students must **critically reflect** on suggestions provided by GenAI and may not accept or submit them without proper review.
- Students may be required to **verbally explain and defend** their work to demonstrate understanding and personal contribution.

!!Important: If there is **reasonable suspicion** the GenAI has been used in a way that violates these rules, this may be classified as an **irregularity**, as defined in the Education and Examination Regulations (EER). Therefore, respect these rules!

0.5 Questions?

Do you have any questions?

- Ask them in class and/or during the Q&A sessions
- Ask them in the Discussion board on Toledo

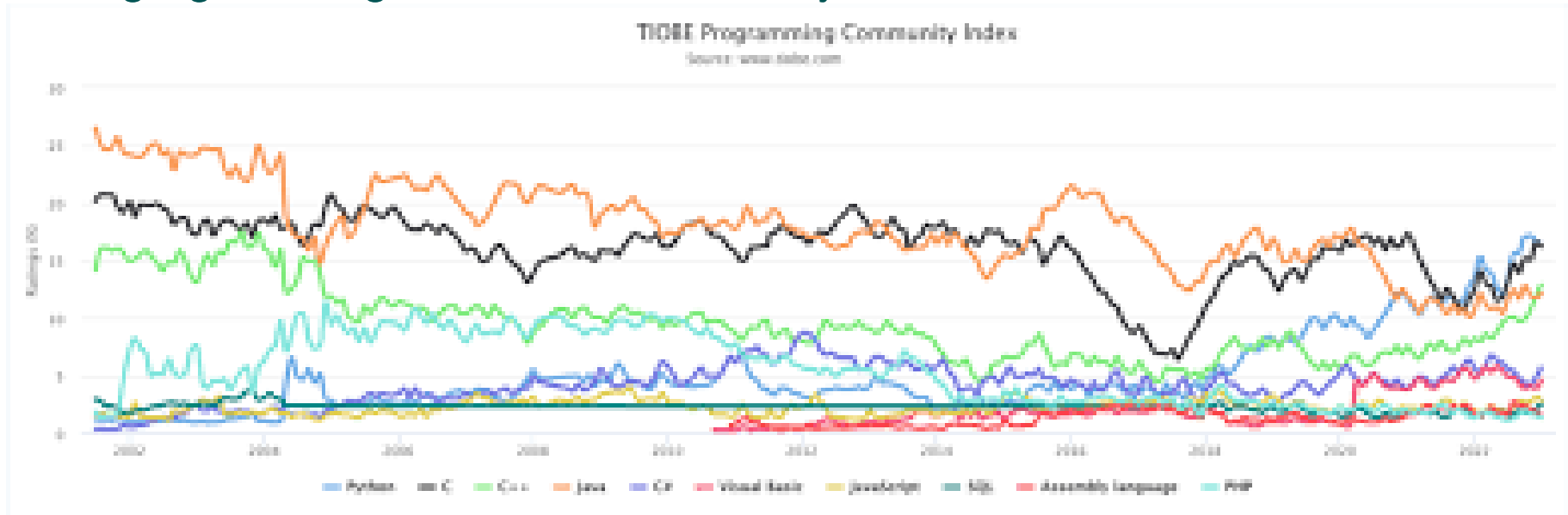
!!Important: Questions related to the course will not be answered by mail! The lecturers of the course are happy to provide you with all necessary info during the official moments and directories.

Part 1:

Python Fundamentals for Data Scientists

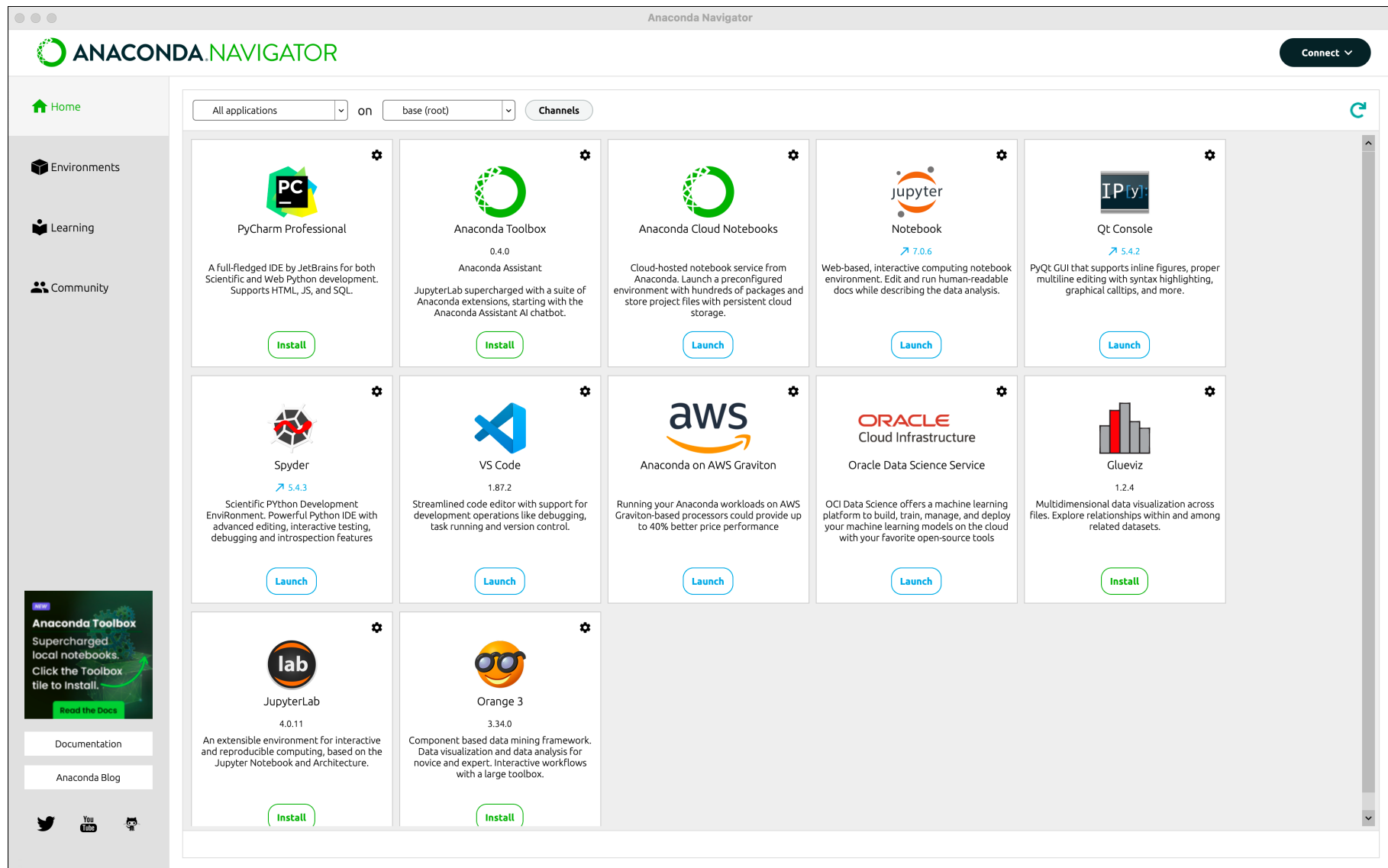
1.1 Python

- Nowadays, Python is considered as one of the most popular programming languages among data scientists, and beyond.



- Generally used when data analysis tasks need to be integrated with web apps or if statistics code needs to be incorporated into a production database.

- While R is mainly used for traditional statistical computing, Python is a good tool for the analysis of extended data science tasks like NLP, computer vision, etc.
- Download & start: **Anaconda**
 - Installation: www.anaconda.com/download
 - Instead of going to the Python.org website to download the required version of Python and then installing the different packages needed, Anaconda, i.e., a package manager, is a quicker introduction
 - Anaconda offers the most useful packages for mathematics, science and engineering
 - In particular, over 300 packages are automatically installed with Anaconda

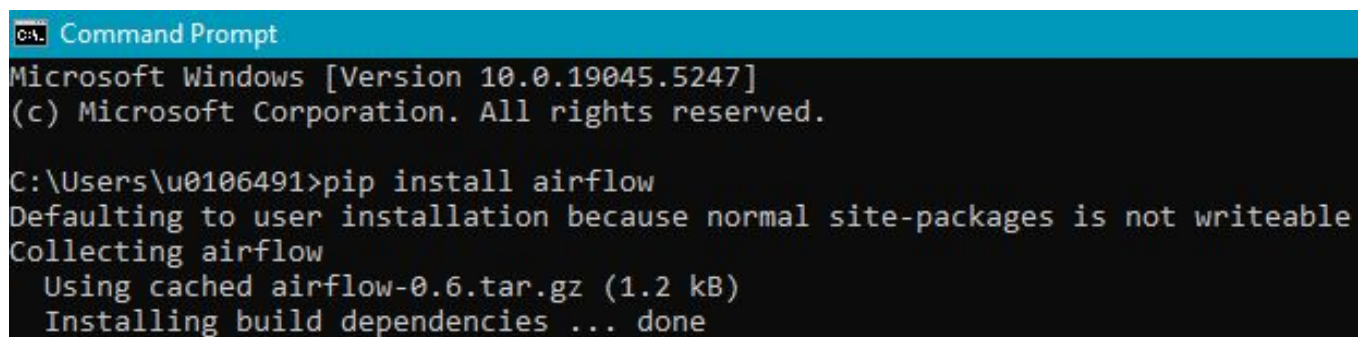


- It can happen that some packages are not installed yet.

```
In [3]: import airflow
```

```
-----  
ModuleNotFoundError                                Traceback (most recent call last)  
~\AppData\Local\Temp\ipykernel_20840\945934463.py in <module>  
----> 1 import airflow  
  
ModuleNotFoundError: No module named 'airflow'
```

- **Solution:** Use the "pip install" command in the command window in order to fetch a particular package and install it on your computer:



```
C:\> Command Prompt  
Microsoft Windows [Version 10.0.19045.5247]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\u0106491>pip install airflow  
Defaulting to user installation because normal site-packages is not writeable  
Collecting airflow  
  Using cached airflow-0.6.tar.gz (1.2 kB)  
  Installing build dependencies ... done
```

→ c:\Users\u0106491\Documents>pip install airflow

- **Important packages:**

- **SciPy and Numpy** for numerical computing
- **Pandas** for easy data processing, cleaning, etc.
- **Matplotlib** to produce graphs
- **Scikit-Learn** for machine learning
- **Statsmodels** for statistical models and unit tests
- **PIL** for basic image importing, manipulation, and exporting

- **MoviePy** is to videos what Pillow is to images. It provides a range of functionality for common tasks associated with importing, modifying and exporting video files
- **Requests** if you application sends any data over HTTP
- **Plotly & Shiny** to produce interactive plots & dashboards
- **PyTorch**: for mathematical operations over tensors. Particular useful for deep learning programming
- **Transformers** for the use of state-of-the-art pre-trained machine learning models. Particular useful in the area of NLP and computer vision

1.2 Writing Code

- When developing and executing code in Python, **IDEs (Integrated Development Environment's)** and **notebooks** play crucial roles.
- An **IDE** is a program dedicated to software development, integrating several tools specifically designed for software development:
 - An editor designed to handle code
 - Build, execution and debugging tools
 - Source control

- In Python, for example, several IDEs are available:
 - Atom
 - Spyder
 - **PyCharm**
 - **Visual Studio (VS) Code**
- In what follows, we will discuss the latter two IDEs, i.e., **PyCharm & VS Code**

1. PyCharm

- Download: www.jetbrains.com/pycharm/
- One of the best and full-featured, dedicated IDE for Python
- Available in both paid (Professional) and free open-source (Community) editions, for Windows, Mac OS X, and Linux platforms
- Supports Python development directly



```
File Edit View Navigate Code Refactor Run Tools VCS Window Help Code - makemore.py
Code makemore.py
Project
  Code C:\Users\u0106491\Documents\LSTAT\Cursussen\
    LICENSE
    makemore.py
    names.txt
    README.md
  External Libraries
  Scratches and Consoles
  CharDataset > _init_()

373     return self.block_size
374
375     def forward(self, idx, targets=None):
376
377         # gather the word embeddings of the previous 3 words
378         embs = []
379         for k in range(self.block_size):
380             tok_emb = self.wte(idx) # token embeddings of shape (b, t, n_embd)
381             idx = torch.roll(idx, 1, 1)
382             idx[:, 0] = self.vocab_size # special <BLANK> token
383             embs.append(tok_emb)
384
385         # concat all of the embeddings together and pass through an MLP
386         x = torch.cat(embs, -1) # (b, t, n_embd * block_size)
387         logits = self.mlp(x)
388
389         # if we are given some desired targets also calculate the loss
390         loss = None
391         if targets is not None:
392             loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)
393
394         return logits, loss
395
396     # -----
397     # Bigram language model
398
399     class Bigram(nn.Module):
400         """
401         Bigram Language Model 'neural net', simply a lookup table of logits for the
402         next character given a previous character.
403         """
404
405     CharDataset > _init_()

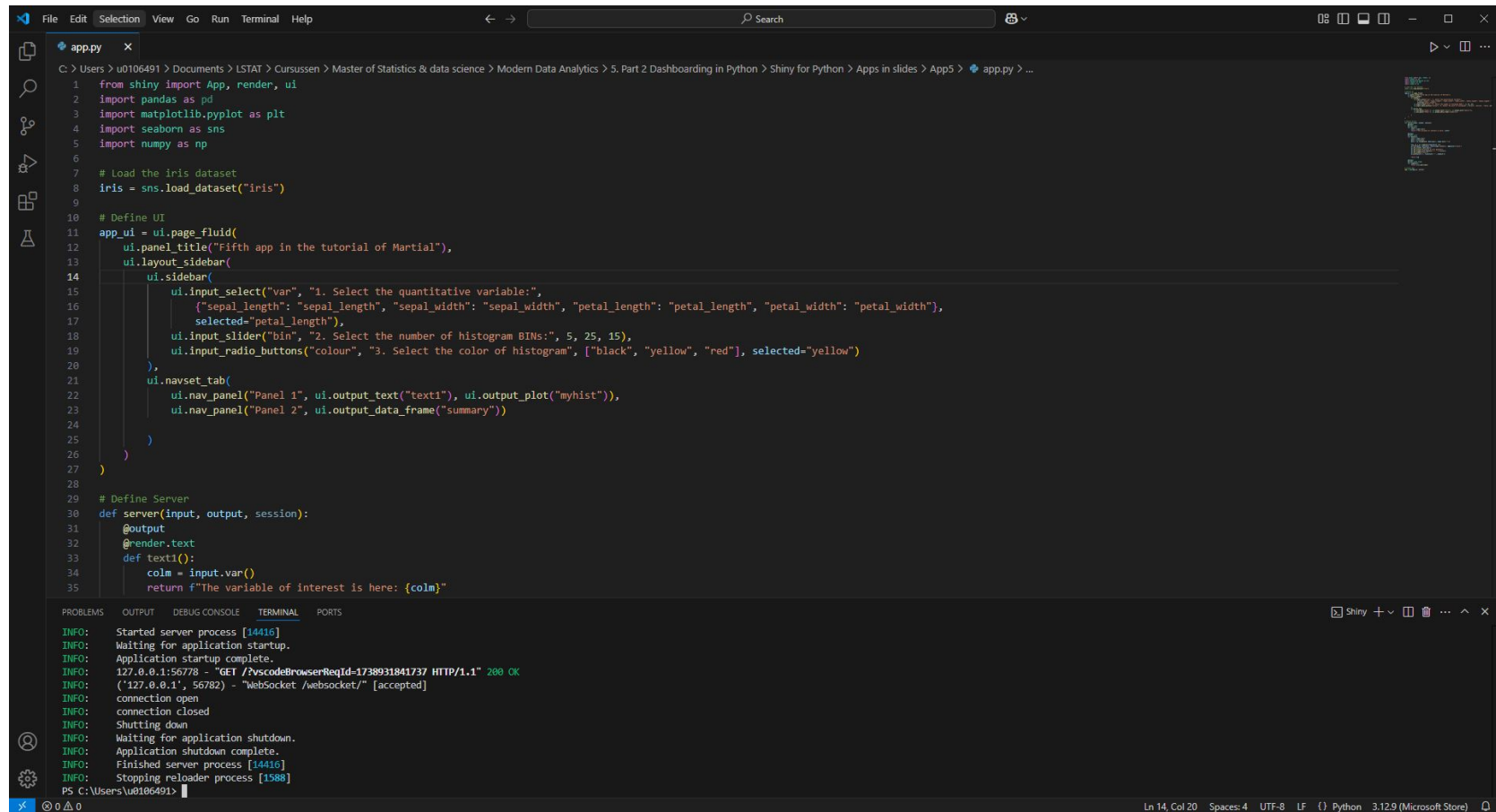
Terminal: Local +
Failed to activate conda environment.
Please open Anaconda prompt, and run 'C:\Users\u0106491\Anaconda3\condabin\conda.bat init powershell' there.
PS C:\Users\u0106491\Documents\LSTAT\Cursussen\Master of Statistics & data science\Text mining\Makemore\Code>

Version Control Python Packages TODO Python Console Problems Terminal Services
Cleaning up skeletons... 512:32 LF UTF-8 4 spaces Python 3.9
```


2. Visual Studio

- Download: code.visualstudio.com/download
- Main difference compared to PyCharm is the user interface:
 - PyCharm provides a comprehensive IDE with a more traditional layout, including various panels and tool windows
 - Visual studio offers a lightweight editor with a minimalistic interface that can be customized extensively with extensions and themes
- More details: <https://bit.ly/3UBx58g>





```
1 from shiny import App, render, ui
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import numpy as np
6
7 # Load the iris dataset
8 iris = sns.load_dataset("iris")
9
10 # Define UI
11 app_ui = ui.page_fluid(
12     ui.panel_title("Fifth app in the tutorial of Martial"),
13     ui.layout_sidebar(
14         ui.sidebar(
15             ui.input_select("var", "1. Select the quantitative variable:",
16                             {"sepal_length": "sepal_length", "sepal_width": "sepal_width", "petal_length": "petal_length", "petal_width": "petal_width"},
17                             selected="petal_length"),
18             ui.input_slider("bin", "2. Select the number of histogram Bins:", 5, 25, 15),
19             ui.input_radio_buttons("colour", "3. Select the color of histogram", ["black", "yellow", "red"], selected="yellow")
20         ),
21         ui.navset_tab(
22             ui.nav_panel("Panel 1", ui.output_text("text1"), ui.output_plot("myhist")),
23             ui.nav_panel("Panel 2", ui.output_data_frame("summary"))
24         )
25     )
26 )
27
28 # Define Server
29 def server(input, output, session):
30     @output
31     @render.text
32     def text1():
33         col = input.var()
34         return f"The variable of interest is here: {col}"
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

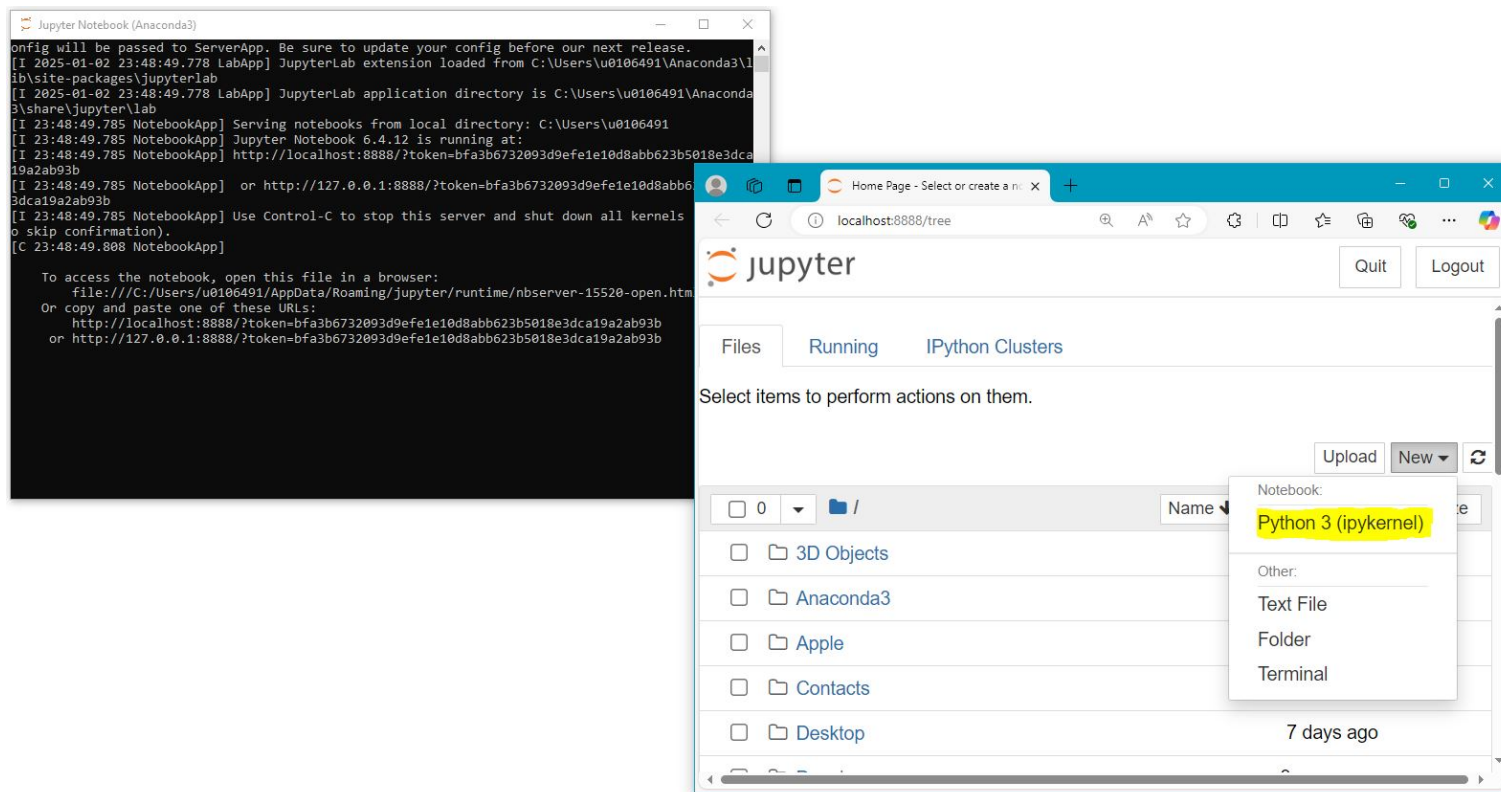
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

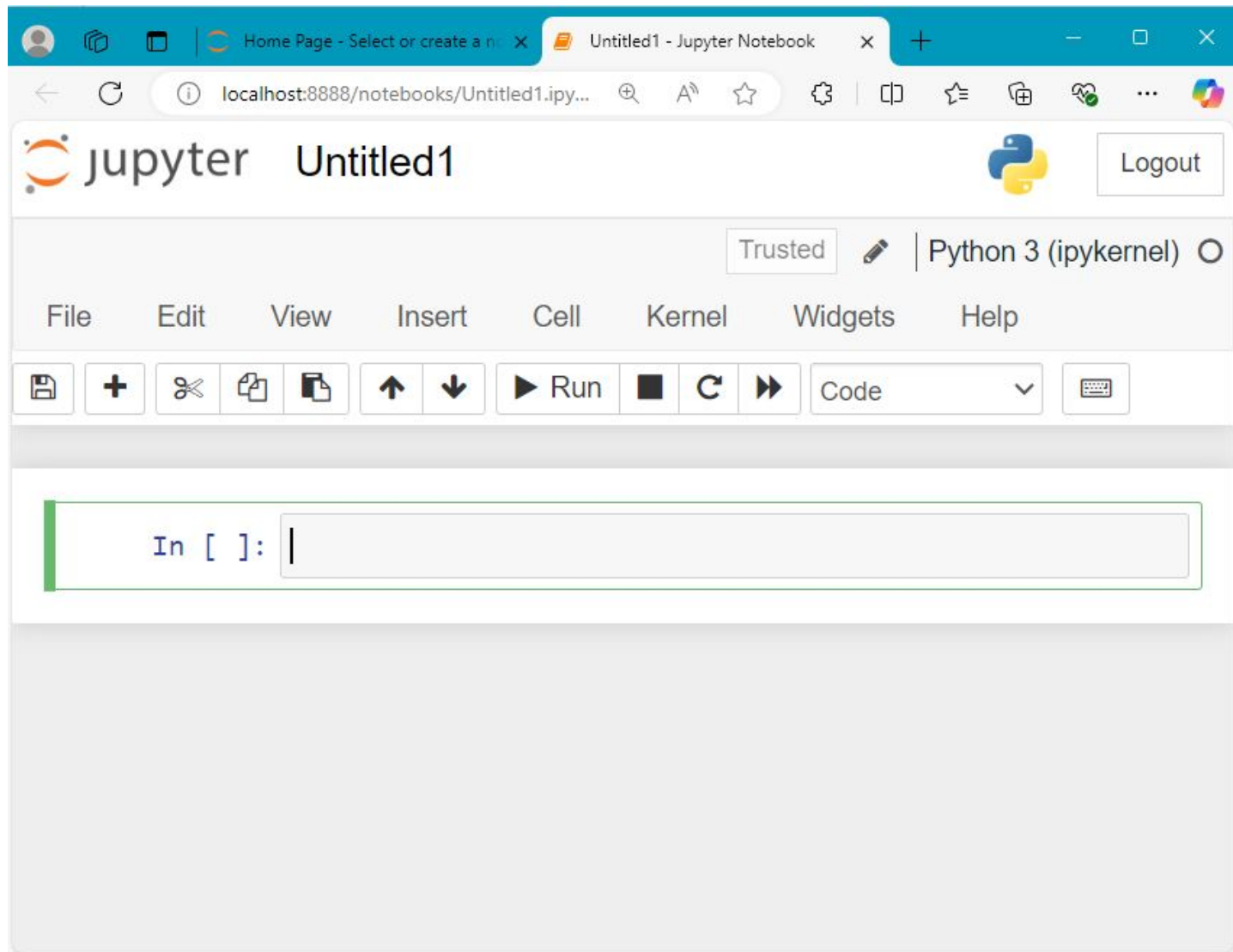
```
INFO: Started server process [14416]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:56778 - "GET /?vscodeBrowserReqId=1738931841737 HTTP/1.1" 200 OK
INFO: ('127.0.0.1', 56782) - "WebSocket /websocket/" [accepted]
INFO: connection open
INFO: connection closed
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [14416]
INFO: Stopping reloader process [1588]
```

PS C:\Users\ue186491>

- Alternative to IDEs, **Jupyter notebooks** can also be considered
 - **Jupyter notebook** = Open-source web application that allows you to create and share documents that contain live code (in Python and other packages) with supporting equations, visualizations, and narrative text
 - Jupyter supports over 100 programming languages, including Python, R, Julia, and Scala
 - Can be shared with others through email, Github, etc.
- In this course, Jupyter notebooks (& VS Code) are used for executing Python code

- To run code in Jupyter notebook, the IPython kernel is used, but additional kernels may be installed





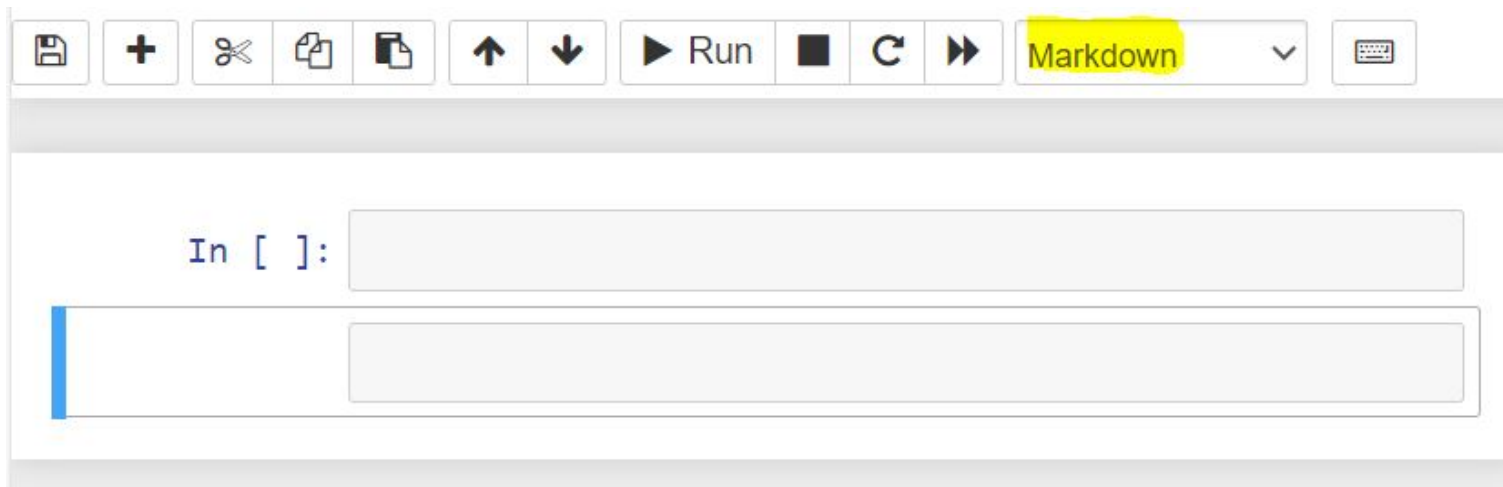
- Different components are available in a Jupyter notebook:
 - **Menu bar** presenting different options that may be used to manipulate the way the notebook functions



- **Toolbar** giving a quick way of performing the most-used operations within the notebook, by clicking on an icon



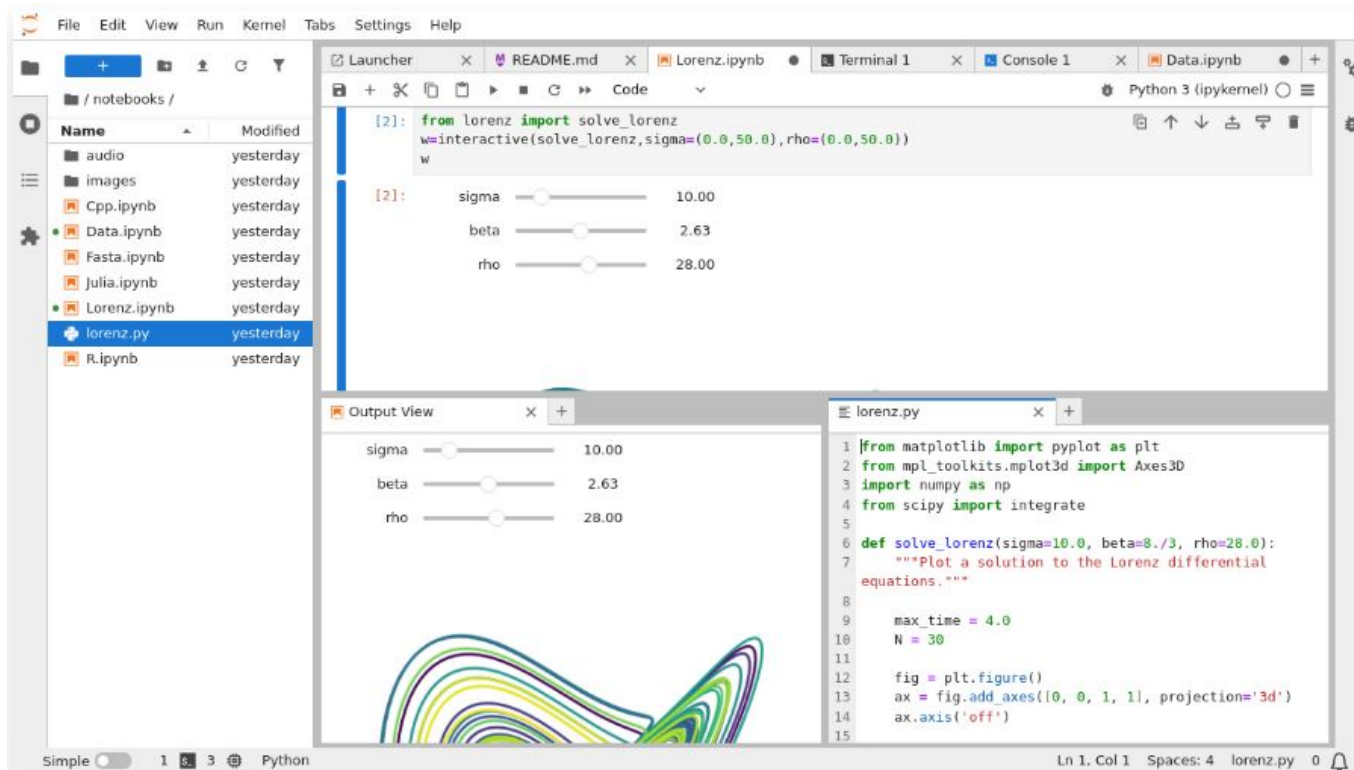
- **Cell**, either "markdown" (comments, titles, etc.) or "code" (Python scripts)



- All actions in the notebook can be performed with a mouse, but keyboard shortcuts are also available for the most common ones:
 - **Shift-Enter** to execute the current cell, show any output, and jump to the next cell below. If this is invoked on the last cell, a new cell will be created below. This is equivalent to clicking the Cell, Run menu item, or the Play button in the toolbar
 - **Ctrl-Enter** to execute the current cell, show the output and stay in the cell
- **Tutorials:**
 - jupyter-notebook.readthedocs.io/en/stable/notebook.html
 - realpython.com/jupyter-notebook-introduction/

- www.datacamp.com/community/tutorials/tutorial-jupyter-notebook
- jupyter-notebook-beginner-guide.readthedocs.io/en/latest/
- www.tutorialspoint.com/jupyter.index.htm
- In addition, extensions can be added to your notebook:
 - Info: towardsdatascience.com/jupyter-notebook-extensions-517fa69d2231
 - Run the following lines in your command line on your PC/Mac:
`pip install jupyter-contrib-nbextensions; jupyter contrib nbextension install --user`

- Alternatively, one could also consider **Jupyterlab**, i.e., the next-generation user interface including notebooks.



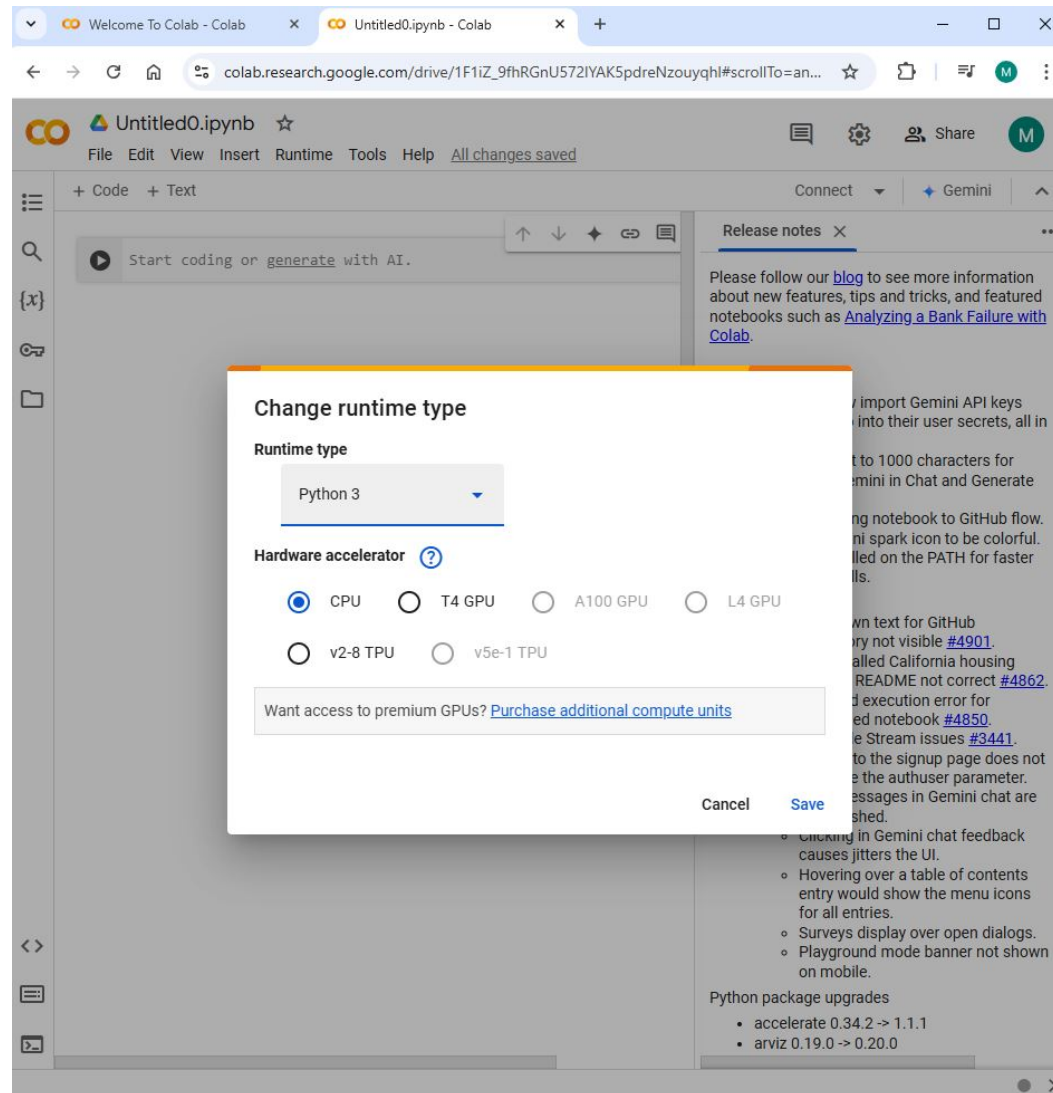
- It has a modular structure, where you can open several notebooks or files (e.g., HTML, Text, etc.) as tabs in the same window. Therefore, it offers more of an IDE-like experience

1.3 Google Colab

- While traditional Jupyter notebooks are versatile and interactive environments that allows users to integrate code, text, equations, and visualizations in a single document (notebook), it runs on local systems
 - **Constraint:** Limited computational power available
- **Question:** What if you have a computer that can't take the workload for, for example, training a machine learning model in Jupyter notebook locally?
- **Possible solution: Google Collaborate (Colab)**
 - **Google Colab** = A free cloud-based platform *provided by Google* that allows users to write and execute Python code collaboratively in a Jupyter notebook environment

- It offers free GPU and TPU access, which could be useful when significant computation power is required
- It is integrated with Google Drive (by mounting; see later), allowing users
 - To access large datasets stored in Google Drive without needing to upload them every time
 - To save their work directly to Google Drive for easy access and sharing
 - To load pre-trained models and other assets directly from Google Drive
- No setup is required, making it convenient for quick coding and collaboration

- Documentation: colab.research.google.com/notebooks/welcome.ipynb
- To get started
 - make sure you have a google account (gmail)
 - then go to this link: colab.research.google.com
- When a new notebook is created, Colab will create Untitled0.ipynb and saves it to your Google Drive in a folder named Colab Notebooks
- You can choice your runtime (CPU, GPU, TPU)



- **Question:** How can a Google Drive be mounted in Google Colab?

- **Answer:**

1. Import the "drive" module from the "google.colab" package. This module provides functions to interact with Google Drive.

```
# Import the library to mount Google Drive
from google.colab import drive
```

2. Use the "mount" function to mount the Google Drive. The "mount" function requires the path where to mount the drive, which is usually '/content/drive'.

```
# Mount the Google Drive at /content/drive
drive.mount('/content/drive')

# Verify by listing the files in the drive
!ls /content/drive/My\ Drive/
```

3. When you run the above code, you will see a prompt with a link to obtain an authorization code. Follow these steps:
- 1) Click on the URL provided in the output
 - 2) Select the Google account you want to use
 - 3) Allow Google Colab to access your Google Drive

4. The Google Drive files can now be accessed as if they were on your local file system. For example, to list the files in the Drive, the "ls" command can be used:

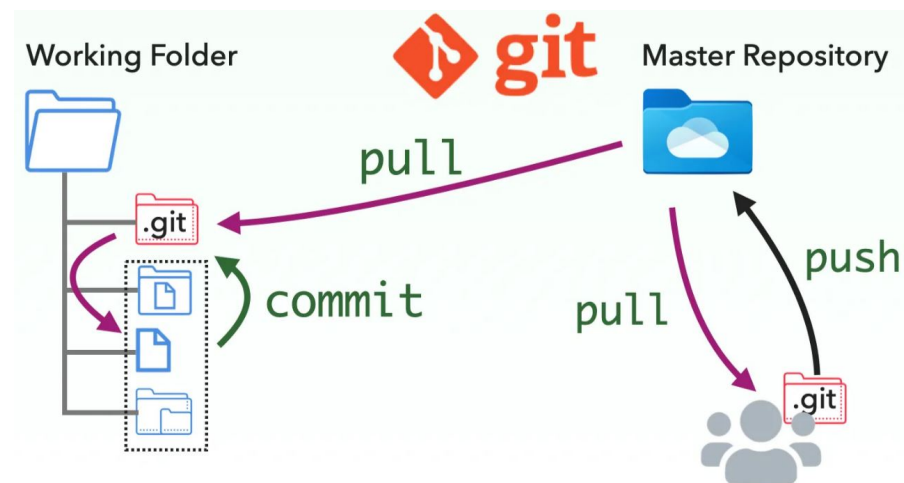
```
# Verify by listing the files in the drive  
!ls /content/drive/My\ Drive/
```


1.4 Git, Github & Hugging Face

- Imagine you are working on a group project, and wishes
 - To split the task (one will work in a separate item)
 - To avoid sending files via email, etc.
 - To track changes
 - To access the code from another device
- **Question:** What kind of system is available that can ensure these demands?

- **Solution: Git and Github**

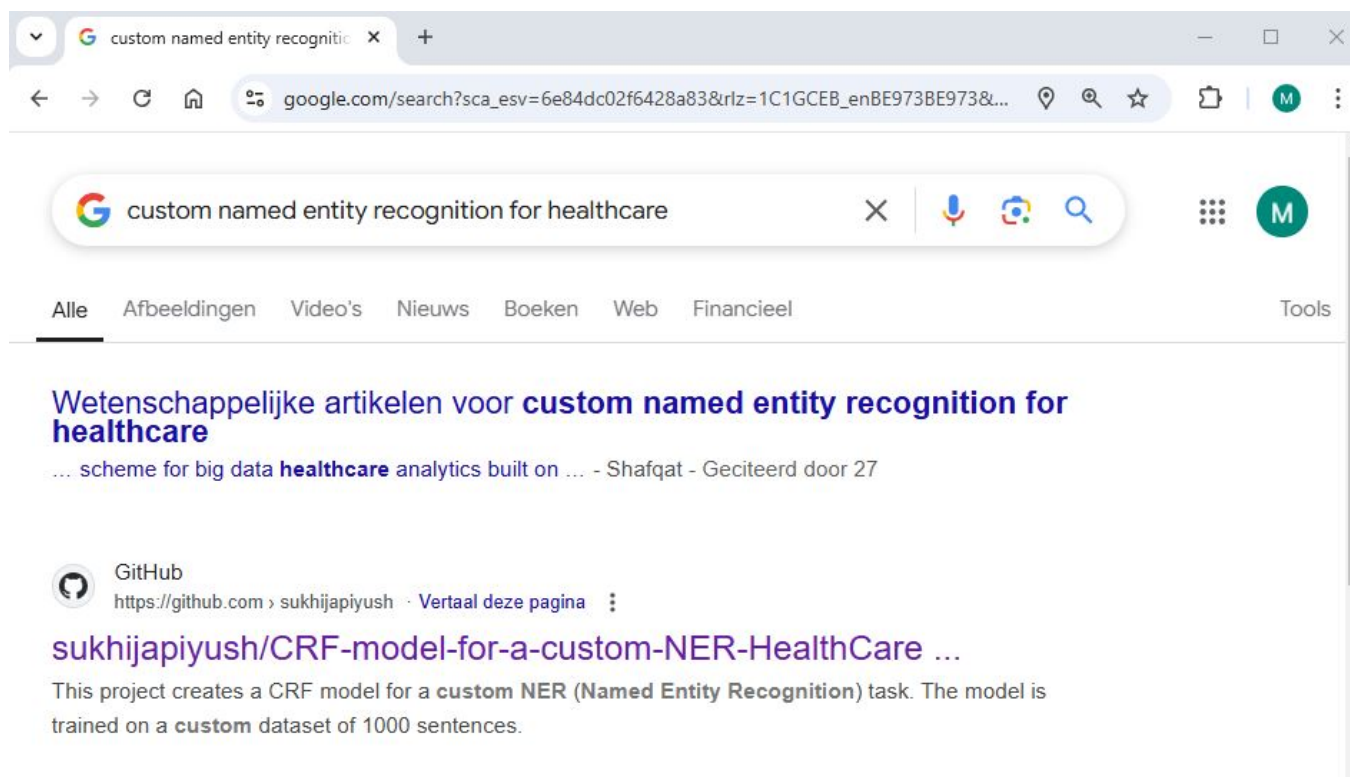
- **Git** was developed in 2005 by Linus Torvalds as open source software for tracking changes in a distributed version control system, i.e., a peer-to-peer type of version control where the complete codebase – including its full version history – is mirrored on every developer's computer.



- Changes to files are tracked between computers, from one developer's workstation to another

- Compared to other version control systems, Git is
 - Responsive
 - Easy to use
 - Inexpensive
- In particular, **Git** distinguish itself from other version control systems by means of its **branching model**
 - **Branching** allows you to create independent local branches in your code
 - In other words, developers can create new ideas, set aside branches for production work, jump back to earlier branches, and easily delete merge, and recall branches at the click of a button

- To keep track of and share Git version control projects outside of your local computer/server, **GitHub**, i.e., a cloud-based database, can be used.
- An individual's Git repositories can be remotely accessed by any authorized person, from any computer, anywhere in the world.



GitHub - sukhijapiyush/CRF-model-for-a-custom-NER-HealthCare-Systems

github.com/sukhijapiyush/CRF-model-for-a-custom-NER-HealthCare-Systems

Sign in

sukhijapiyush /

CRF-model-for-a-custom-NER-HealthCare-Systems Public

Code Issues Pull requests Actions Projects Security Insights

master

Go to file

<> Code

About

Named Entity Recognition for HealthCare Data using Custom CRF model and predict disease pf patients based on complaints

nlp crf python3 spacy

named-entity-recognition

nlp-machine-learning

spacy-nlp-ner spacy-ner

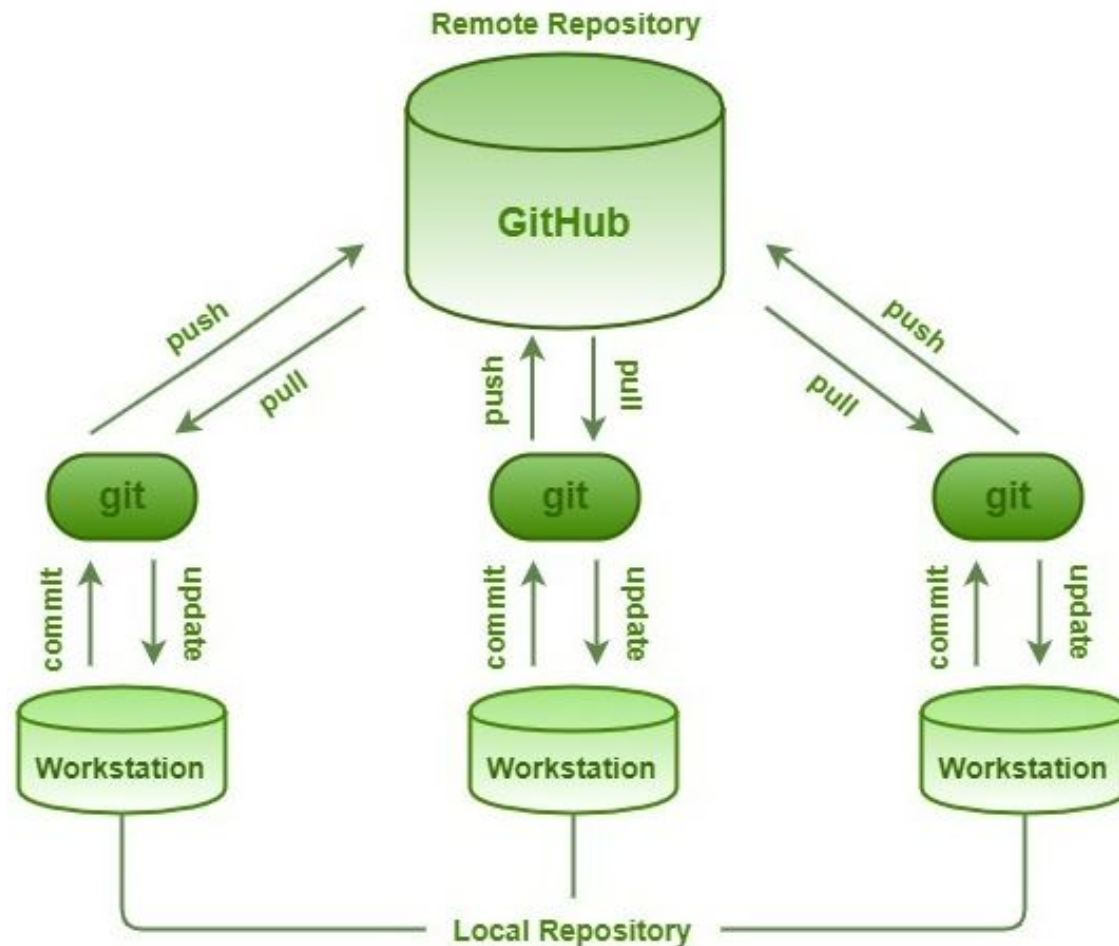
spacy-transformers

Readme

Apache-2.0 license

sukhijapiyush	Update README.md	7db5ab7 · 2 years ago
.github/ISSUE_TEMPLATE	Update issue templates	2 years ago
CODE_OF_CONDUCT.md	Create CODE_OF_CONDUCT.md	2 years ago
CONTRIBUTING.md	Create CONTRIBUTING.md	2 years ago
First solution.ipynb	Updating Notebooks	2 years ago
LICENSE	Create LICENSE	2 years ago
README.md	Update README.md	2 years ago

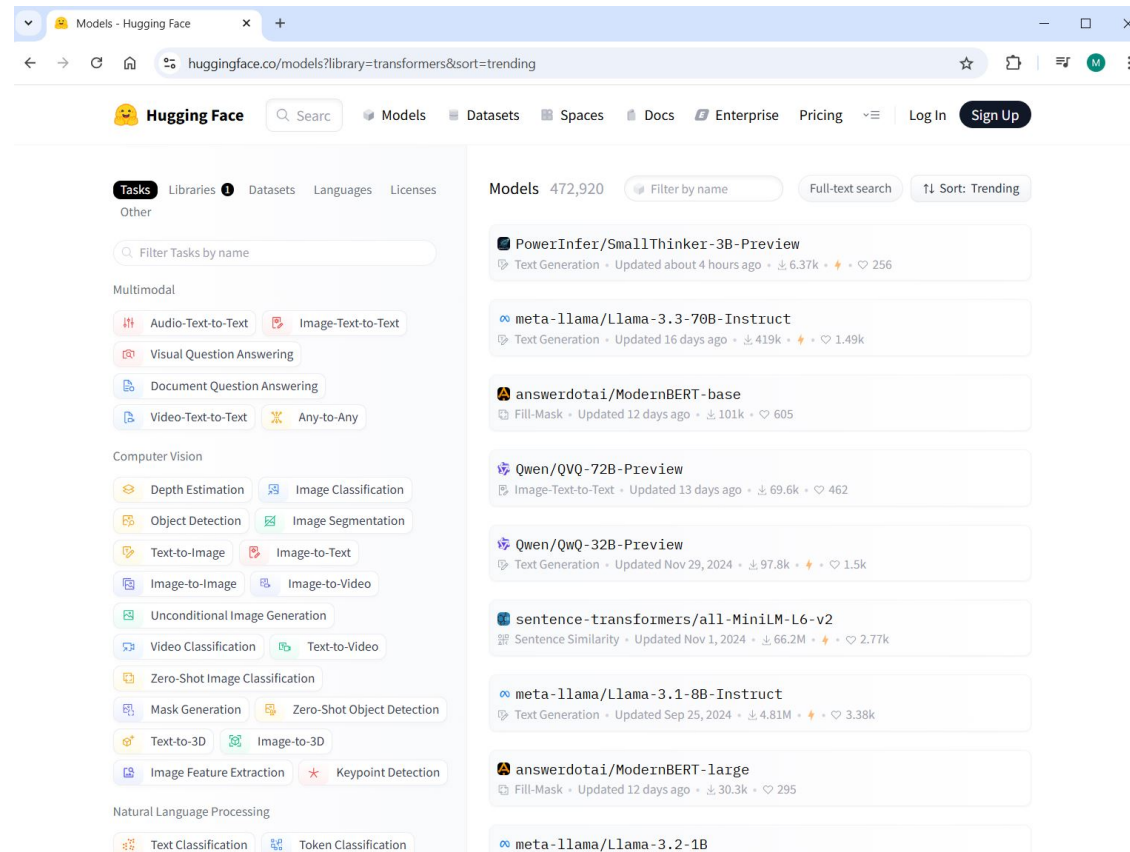
- Through Github, you can share your code with others, giving them the power to make revisions or edits on your various Git branches



- As changes are introduced, new branches are created, allowing the team to continue to revise the code without overwriting each other's work. These branches are like copies, and changes made on them do not reflect in the main directories on other users' machine unless users choose to push/pull the changes to incorporate them
- It is an essential tool to know nowadays for data science/related jobs
- Useful tutorial: [youtube.com/watch?v=mj-qvsxPHpY](https://www.youtube.com/watch?v=mj-qvsxPHpY)
- In order to use a GitHub repository, use the following code:

```
> git clone https://github.com/zspatter/weather-forecast.git
```
- This could be useful for considering models where the source code is available on GitHub, for example

- Another useful and popular community platform is **Hugging Face**
- It is a powerhouse for natural language processing, computer vision and reinforcement learning, offering state-of-the-art open-source pre-trained (language) models like BERT, GPT, and many more



- It incorporates the state-of-the-art Transformers library, developed by Hugging Face, and can be incorporated in your Python code as follows:

```
In [4]: from transformers import pipeline
```

```
In [5]: classifier = pipeline("text-classification",  
                             model="distilbert/distilbert-base-uncased-finetuned-sst-2-english")  
  
classifier("This is a great course")
```

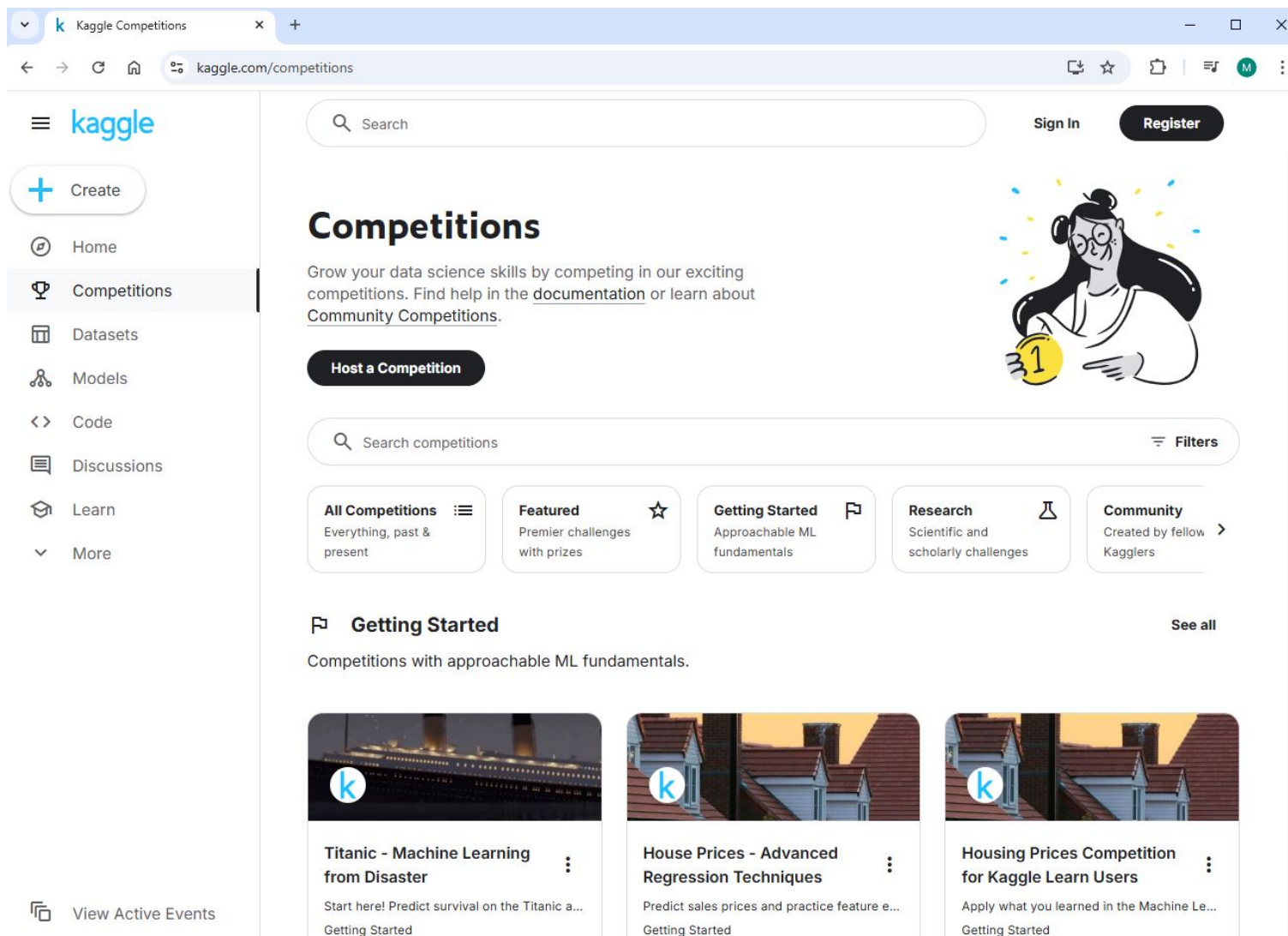
```
Out[5]: [{'label': 'POSITIVE', 'score': 0.9998713731765747}]
```

- In addition to pre-trained models, Hugging face also includes datasets, research papers & a community place for data scientists and others.
- In this course, the following GitHub repository is used for sharing Python code:

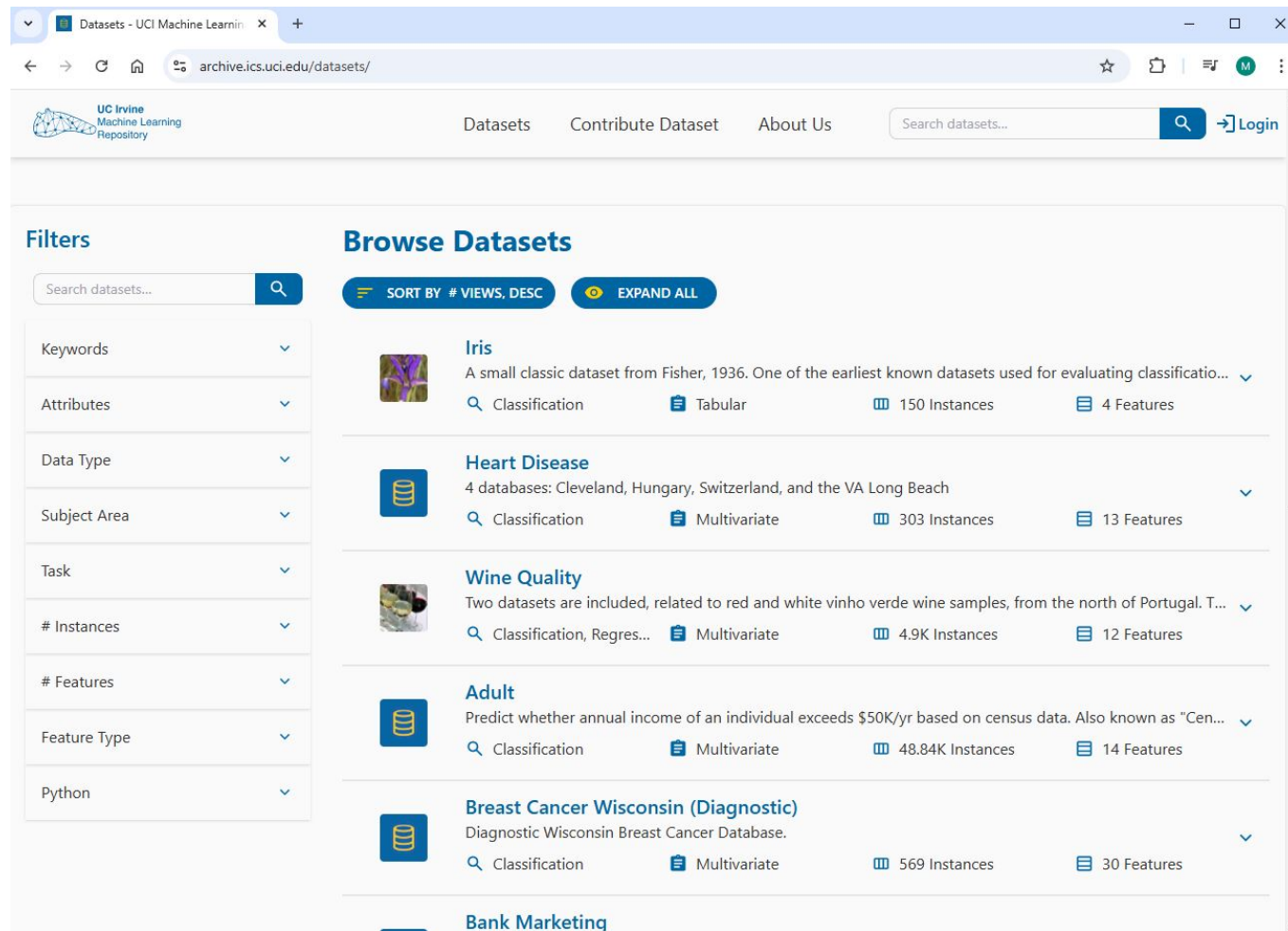
github.com/GregCollab/G0Z39A

1.5 Kaggle and UCI Machine Learning Repository

- **Question:** Does there exist other platforms, in addition to HuggingFace, that offer free datasets?
- **Answer:** Kaggle and UCI Machine Learning Repository
 - **Kaggle** is a data science competition platform and online community for data scientists and machine learning practitioners under Google.
 - **Kaggle** enables users to
 - find and publish datasets
 - explore and build models in a web-based data science environments
 - work with other data scientists
 - enter competitions to solve data science challenges



- In addition, the **UC Irvine Machine Learning Repository** is available, offering datasets for *classification, regression, clustering and others*

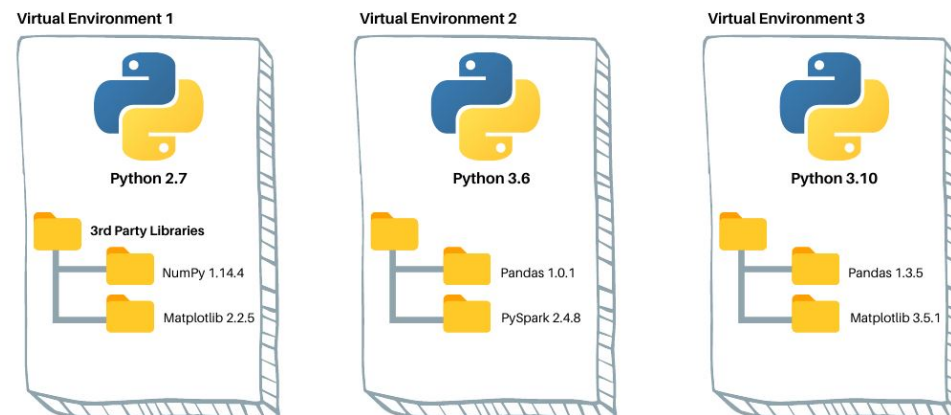


1.6 Virtual Environments

- Assume you are working on app A, using your system installed Python and you pip install packageX version 1.0 to your global Python library.
- After a while, you switch to project B on your local machine, and you install the same package packageX but version 2.0, which has some breaking changes between version 1.0 and 2.0.
- When going back to run appA, it can occur that your app does not run, and you get a lot of errors
- **Question:** How can you solve this issue?

- **Answer: Virtual Environments**

- **Virtual environment (VI)** = Python environment s.t. the Python interpreter, libraries and scripts installed into it are isolated from those installed in other virtual environments, and (by default) any libraries installed in a "system" Python, i.e., one which is installed as part of your operating system.
- As implication, a project in a defined VI becomes its own self contained application, independent of the system installed Python and its modules



- A new VI has its own pip to install libraries, its own libraries folder, and its own Python interpreter for the Python version you used to active the environment.
- To set up a Python environment, the virtualenv package can be used.

Steps:

1. Install virtualenv in your command line:

```
> pip install virtualenv
```

2. Create a new project folder, cd to the project folder in your terminal, and run the following command:

```
> python<version> -m venv <virtual-environment-name>
```

Example:

```
> python -m venv myenv
```

3. To activate the VI, use the following command:

```
> myenv\Scripts\activate
```


4. To install all packages, it is best to generate a text file listing all your project dependencies:

```
> pip freeze > requirements.txt
```

5. Instead of installing each dependency one by one, all dependencies listed in the requirements.txt file can be installed directly using the following command:

```
> pip install -r requirements.txt
```

6. To deactivate the virtual environment, use the following command:

```
$ deactivate
```

- **Documentation:**

- bit.ly/3h4f8ty
- bit.ly/3r5KUL4

1.7 Object Oriented Programming

- Python is an **Object Oriented Programming (OOP)** language
- **OOP** is a style of programming used to build modular, maintainable, and scalable applications
- It is a way of organizing code that uses classes (representing concepts) and objects (instances of classes) to represent real-world entities and their behavior
 - Classes can be thought of as a 'blueprint' for objects. These can have their own attributes (characteristics they possess), and methods (actions they perform)

Example: Consider the class "Dog"

- Dogs usually have a name and age, i.e., instance attributes
- Dogs can also bark, i.e., a method

```

class Dog:
    # Class attribute
    species = "Golden retriever"

    def __init__(self, name, age):
        # Instance attributes
        self.name = name
        self.age = age

    def bark(self):
        # Method
        print("bark bark!")

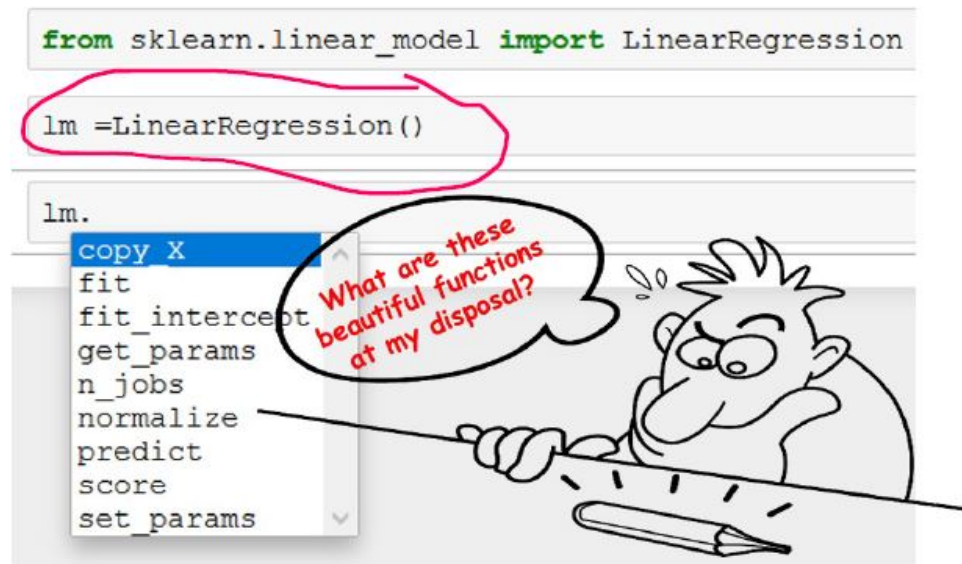
# Creating an object of the Dog class
dog1 = Dog("Marcel", 2)
dog2 = Dog("Charlie", 5)

# Access class and instance variables
print(dog2.species)
print(dog1.name)
dog1.bark()

```

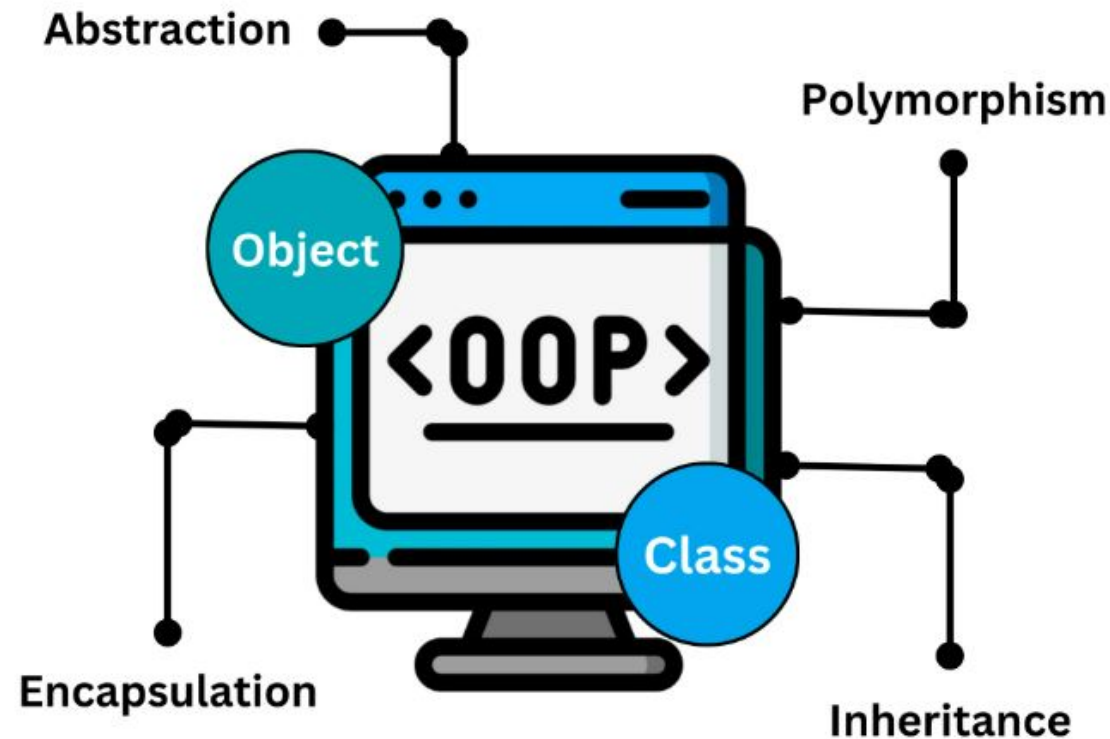
- **Remark 1:** All classes have a function called `__init__()`, which is always executed when the class is being initiated. It is used to assign values to object properties, or other operations that are necessary to do when the object is being created.
- **Remark 2:** *self* parameter is a reference to the current instance of the class. It allows us to access the attributes and methods of the object.

- Key data science libraries such as pandas, numpy, and scikit-learn all heavily rely on OOP
- **Question:** How can you see this?



- **Answer:** In scikit-learn, for example, a regression model is as an instance of a class, and it has a `fit()` method to train your machine learning model or a `predict()` method for prediction

- OOP allows objects to interact with each other using four basic principles:



- These four principles enables objects to communicate and collaborate to create powerful applications

- **Tutorials:**

- bit.ly/38AE7Rj

- bit.ly/3ryOqOa

- bit.ly/3poTrHg

- Let's get convinced with a simple example, i.e., build a linear regression model (as you always do) or using OOP

- **Linear_regression.ipynb**